

ECOLE NATIONALE SUPERIEURE POLYTECHNIQUE DE YAOUNDE

Departement du Genie Informatique

Unite d'Enseignement : Intelligence Artificielle et Applications – ANI-IA 4



Rapport de Projet

Systemes de Recommandation par
Reseaux de Neurones sur Graphes
LightGCN – Light Graph Convolutional Network

Rapport de Projet · Dans le cadre du cours Intelligence Artificielle et Applications

Presente par :

BILOA ABADJECK

Valery Paolo Stelane

Matricule : **22P009**

Classe : **ANI-IA 4**

Encadre par :

M. OMGBA BITHA Junior

Module :

**Intelligence Artificielle
et Applications**

Repository GitHub : <https://github.com/paolo-88-hub/projet1-lightgcn-AIA-4>

Ecole Nationale Supérieure Polytechnique de Yaoundé | Année académique
2025/2026

Resume

Ce rapport presente la conception, l'implementation et l'evaluation d'un **systeme de recommandation de films** base sur le modele **LightGCN** (Light Graph Convolutional Network), realise dans le cadre de l'Unite d'Enseignement *Intelligence Artificielle et Applications* (ANI-IA 4) a l'Ecole Nationale Superieure Polytechnique de Yaounde.

Le projet exploite le dataset **MovieLens ml-latest-small** (100 836 interactions brutes, 610 utilisateurs, 3 650 films). Apres filtrage, 89 664 interactions sont conservees pour l'entrainement. LightGCN modelise les interactions sous forme d'un **graphe biparti normalise** et propage les embeddings sur **K=3 couches** pour capturer les collaborations de haut ordre. L'optimisation est realisee par la **perte BPR** (Bayesian Personalized Ranking) via l'optimiseur Adam sur 100 epoques.

Les resultats obtenus sont : **Recall@10 = 0,0754** et **NDCG@10 = 0,0308**, avec une convergence de la perte BPR de 0,676 a 0,150 (−78%).

Au-dela des livrables initialement remis, une **contribution personnelle supplementaire** a ete developpee apres reflexion : un **dashboard interactif temps reel** (Flask + SocketIO) permettant de visualiser l'ensemble du pipeline en direct depuis le navigateur, avec courbes de convergence, metriques live et conclusion automatique. Cette interface n'etait pas demandee dans les livrables du projet initial – elle constitue une innovation ajoutee pour rendre le travail plus explicite et demonstrable.

Mots-cles : LightGCN, Graph Convolutional Network, Systeme de recommandation, Filtrage collaboratif, BPR Loss, Embeddings, MovieLens, PyTorch, Dashboard temps reel, Flask-SocketIO.

Contents

Resume	1
1 Introduction	8
1.1 Contexte	8
1.2 Objectifs du projet	8
1.3 Organisation du rapport	9
2 Etat de l'Art	10
2.1 Systemes de recommandation	10
2.2 De la factorisation de matrices aux GCN	10
2.3 LightGCN – Simplification elegante	11
2.3.1 Formulation mathematique	11
2.3.2 Perte BPR	11
2.3.3 Avantages de LightGCN	11
2.4 Dataset MovieLens	12
3 Methodologie et Environnement Technique	13
3.1 Environnement d'execution	14
3.2 Livrables Attendus et Contribution Supplementaire	14
3.2.1 Livrables initialement prevus	14
3.2.2 Contribution personnelle supplementaire – Dashboard Temps Reel	15
3.3 Arborescence du projet	16
3.4 Pipeline de traitement	17
3.5 Livrables du projet	17
3.5.1 Livrables initiaux remis	17
3.5.2 Contribution personnelle supplementaire	18
3.6 Hyperparametres du modele	18
4 Implementation	19
4.1 Fichier <code>src/config.py</code> – Configuration centrale	19

4.2	Fichier <code>src/dataset.py</code> – Chargement des donnees	20
4.3	Fichier <code>src/graph.py</code> – Graphe biparti	20
4.4	Fichier <code>src/model.py</code> – Architecture LightGCN	21
5	Mode d’Emploi – Installation et Lancement	22
5.1	Prerequis	22
5.2	Installation complete pas a pas	22
5.2.1	Etape 1 – Naviguer vers le projet	22
5.2.2	Etape 2 – Creer et activer l’environnement virtuel	22
5.2.3	Etape 3 – Installer les dependances	23
5.2.4	Etape 4 – Telecharger MovieLens	23
5.3	Lancement	23
5.3.1	Option A – Pipeline en ligne de commande	23
5.3.2	Option B – Dashboard temps reel (recommande)	24
6	Resultats et Interfaces	25
6.1	Interface du Dashboard – Etat initial	25
6.2	Phase 1 et 2 – Chargement dataset et graphe	25
6.3	Phase 3 – Entrainement en direct	26
6.4	Progression de l’entrainement – Epoques 20 a 40	27
6.5	Progression avancee – Epoque 95	28
6.6	Pipeline termine – Resultats finaux	29
6.7	Section Analyse et Conclusion automatique	30
6.8	Resultats numeriques obtenus	30
6.9	Contribution personnelle – Pourquoi un dashboard ?	31
7	Difficultes Rencontrees et Solutions	32
7.1	Probleme 1 – Python 3.14 incompatible CUDA	32
7.1.1	Description	32
7.1.2	Tentatives et solution	32
7.2	Probleme 2 – Espace manquant dans la commande <code>pip</code>	32
7.3	Probleme 3 – <code>main.py</code> dans le mauvais dossier	33
7.4	Probleme 4 – <code>n_iter</code> deprecie dans scikit-learn	33
7.5	Probleme 5 – Sparse tensor warning PyTorch	33
7.6	Probleme 6 – Fichier lourd sur GitHub	33
7.7	Recapitulatif des difficultes	34
8	Conclusion et Perspectives	35
8.1	Bilan du projet	35
8.2	Apport personnel et innovation	35

8.3	Reponse aux objectifs	36
8.4	Perspectives d'amelioration	36
Remerciements		37
References bibliographiques		38
Glossaire		39
A Commandes PowerShell Completes		41
B Fichier requirements.txt		43

List of Figures

3.1	Pipeline d'entraînement LightGCN – 4 phases	17
6.1	Dashboard LightGCN – Etat initial avant lancement. Vue d'ensemble : badges ANI-IA 4, bouton “Lancer LightGCN”, 6 cartes de parametres, 4 phases d'execution en attente et zones de courbes vides.	25
6.2	Dashboard en cours – Phases 1 et 2 terminees. Dataset : 610 users, 3650 items, 89 664 interactions. Graphe : matrice 4260x4260, normalisation symetrique. Phase 3 (Entrainement BPR) en cours d'initialisation. . . .	26
6.3	Section “Entrainement en direct” – Zoom sur les metriques live et le journal d'execution. Visible : progression 0/100 epoques, cartes BPR Loss / Recall@10 / NDCG@10 en attente de donnees, journal affichant les messages d'initialisation.	27
6.4	Vue VS Code + Dashboard a l'epoque 20. Gauche : code <code>dashboard.py</code> ouvert dans VS Code avec le terminal PowerShell affichant les logs en direct. Droite : dashboard montrant BPR Loss=0.3128, Recall@10=0.0525, NDCG@10=0.0259. Progression 20/100.	27
6.5	Dashboard a l'epoque 40 – BPR Loss=0.2451, Recall@10=0.0541, NDCG@10=0.0267. Les courbes Chart.js montrent la convergence de la loss (rouge) et la progression des metriques Recall@10 (vert) et NDCG@10 (orange). Progression 40/100.	28
6.6	Dashboard a l'epoque 95/100 – BPR Loss=0.1546, Recall@10=0.0721, NDCG@10=0.0309. La courbe BPR Loss (rouge) montre une convergence nette de 0.7 vers 0.15. La courbe Recall@10 (verte) progresse regulierement. Journal : epoques 20 a 90 avec marquage “meilleur”. . .	28
6.7	Dashboard complet apres 100 epoques. Les 4 phases affichent TERMINER a 100%. Metriques finales : BPR Loss=0.1497, Recall@10=0.0754, NDCG@10=0.0308. Courbes completes sur 100 epoques. Le modele optimal a ete sauvegarde a l'epoque 100.	29

6.8

Section “Analyse et Conclusion” generee automatiquement par le dash-
board a la fin du pipeline. Resume contextualise : interpretations
des metriques Recall@10=0.0754 et NDCG@10=0.0308, reduction BPR
Loss de 77.9%, progression Recall de 64.3%. Trois cartes : points forts,
limite principale, piste d’amelioration.

30

List of Tables

2.1	Caracteristiques du dataset MovieLens ml-latest-small	12
3.1	Methodes et Technologies Employees – Projet 1	14
3.2	Hyperparametres de LightGCN	18
6.1	Evolution des metriques au cours de l’entrainement	30
7.1	Recapitulatif des difficultes et solutions	34
8.1	Reponse aux objectifs initiaux	36

Introduction

1.1 Contexte

Les systemes de recommandation constituent aujourd'hui un pilier fondamental des plateformes numeriques mondiales. De Netflix a Spotify, la capacite a proposer du contenu personnalise repose sur des algorithmes d'intelligence artificielle capables d'exploiter les interactions passees pour predire les preferences futures.

Le defi central est le *filtrage collaboratif* : comment inferer qu'un utilisateur aimera un article qu'il n'a jamais vu, en s'appuyant uniquement sur les interactions historiques d'une communaute ? Les approches classiques (factorisation de matrices) ont montre leurs limites face aux donnees eparses et aux relations complexes de haut ordre.

LightGCN (He et al., 2020) repond a ces limitations en modelisant explicitement la structure du graphe d'interactions, avec une propagation pure et elegante ne necessitant aucune transformation ni activation non lineaire.

1.2 Objectifs du projet

1. Comprendre les fondements theoriques des Graph Convolutional Networks et leur simplification via LightGCN.
2. Implementer LightGCN *from scratch* en PyTorch sans librairie tierce dediee aux graphes.
3. Entrainer le modele sur MovieLens avec la perte BPR et l'optimiseur Adam.
4. Evaluer les performances avec Recall@10 et NDCG@10.
5. Deployer un **dashboard interactif temps reel** (Flask + SocketIO) montrant le pipeline en direct.

6. Generer des recommandations Top-10 personnalisées.

1.3 Organisation du rapport

Ce rapport suit la structure suivante : Chapitre 2 — Etat de l'art. Chapitre 3 — Methodologie et environnement. Chapitre 4 — Implementation. Chapitre 5 — Mode d'emploi complet. Chapitre 6 — Resultats et interfaces. Chapitre 7 — Difficultes et solutions. Chapitre 8 — Conclusion.

Etat de l'Art

2.1 Systemes de recommandation

Les systemes de recommandation se divisent en trois familles principales :

Filtrage base sur le contenu Recommande des items similaires a ceux deja apprecies, en exploitant leurs caracteristiques intrinseques.

Filtrage collaboratif Exploite les interactions collectives pour trouver des utilisateurs similaires. C'est l'approche de LightGCN.

Approches hybrides Combinent les deux methodes pour beneficier de leurs avantages respectifs.

2.2 De la factorisation de matrices aux GCN

La factorisation de matrices (MF) fut longtemps l'etat de l'art. Elle represente chaque utilisateur et article par un vecteur latent et predit l'interaction par produit scalaire. Cependant, MF souffre de :

- L'incapacite a capturer des relations de haut ordre.
- Une performance degradee sur des donnees tres eparses.
- L'absence de modelisation explicite de la structure du graphe.

NGCF (Wang et al., 2019) a propose de propager les informations sur le graphe biparti utilisateur-item. Mais NGCF introduit des transformations et activations superflues.

2.3 LightGCN – Simplification elegante

LightGCN simplifie radicalement NGCF en ne conservant que l'essentiel : la propagation pure des embeddings.

2.3.1 Formulation mathématique

Soit $\tilde{A} = D^{-1/2}AD^{-1/2}$ la matrice d'adjacence normalisee du graphe biparti. La propagation a la couche k :

$$E^{(k)} = \tilde{A} \cdot E^{(k-1)} \quad (2.1)$$

L'embedding final est la moyenne ponderee de toutes les couches :

$$E^* = \frac{1}{K+1} \sum_{k=0}^K E^{(k)} \quad (2.2)$$

Le score de prediction pour la paire (u, i) :

$$\hat{y}_{ui} = (e_u^*)^\top \cdot e_i^* \quad (2.3)$$

2.3.2 Perte BPR

$$\mathcal{L}_{\text{BPR}} = - \sum_{(u,i,j)} \ln \sigma(\hat{y}_{ui} - \hat{y}_{uj}) + \lambda \|E^{(0)}\|^2 \quad (2.4)$$

ou (u, i, j) est un triplet avec i un item positif et j negatif.

2.3.3 Avantages de LightGCN

- Moins de parametres : seuls les embeddings $E^{(0)}$ sont appris.
- Interprétabilité : schema mathematique clair.
- Performance superieure a NGCF sur les benchmarks standards.
- Scalabilité grace aux matrices sparse.

2.4 Dataset MovieLens

Table 2.1: Caracteristiques du dataset MovieLens ml-latest-small

Caracteristique	Valeur	Remarque
Nombre de ratings bruts	100 836	Fichier ratings.csv
Utilisateurs	610	Apres filtrage
Films total	9 742	Catalogue complet
Films apres filtrage	3 650	Items retenus
Interactions train	89 664	Leave-one-out
Interactions test	610	1 par utilisateur
Periode	1996–2018	Historique long

Methodologie et Environnement Technique

3.1 Environnement d'exécution

Table 3.1: Methodes et Technologies Employees – Projet 1

Composant	Detail
Livrables initiaux remis	
Modele	LightGCN (He et al., 2020)
Framework IA	PyTorch 2.12.0 + torch_sparse (CPU)
Langage	Python 3.14.5
Dataset	MovieLens ml-latest-small
Representation	Graphe biparti sparse (COO format)
Fonction de perte	BPR (Bayesian Personalized Ranking)
Optimiseur	Adam (lr=0.001)
Metriques	Recall@10, NDCG@10
Visualisation	Matplotlib, Seaborn, t-SNE
Environnement initial	VS Code + PowerShell, Windows 11
Innovation personnelle ajoutée – Dashboard Temps Reel	
Interface web	Flask 3.x + Flask-SocketIO 5.x
Communication	WebSocket (protocole SocketIO)
Graphiques live	Chart.js 4.4
Progression temps reel	Barres animees + metriques live
Conclusion automatique	Generee dynamiquement en fin de pipeline
Port	http://localhost:5000
Processeur	Intel Core i5 (CPU uniquement)
Repository	https://github.com/paolo-88-hub/projet1-lightgcn-AIA-4

Note sur le GPU

Python 3.14 n'étant pas encore supporte par PyTorch pour CUDA, tout l'entrainement s'est deroule sur CPU Intel Core i5. Duree totale : environ 15–20 minutes pour 100 epoques.

3.2 Livrables Attendus et Contribution Supplemen- taire

3.2.1 Livrables initialement prevus

Les livrables definis dans le cadre du projet ANI-IA 4 etaient :

- Code source complet et commente (Python)
- Graphiques de convergence (courbe de perte par epoque)
- Tableau de resultats Recall@10 et NDCG@10
- Visualisation t-SNE des embeddings
- Exemples de recommandations Top-10 pour plusieurs utilisateurs

3.2.2 Contribution personnelle supplementaire – Dashboard Temps Reel

Innovation personnelle – non prevue dans les livrables initiaux

Après reflexion et a l'issue du developpement du projet, j'ai concu et implemente par **initiative propre** un **dashboard interactif temps reel** qui ne figurait pas dans les livrables demandes. Cette contribution supplementaire constitue une amelioration significative de la presentation et de la comprehension du travail realise.

Pourquoi ce dashboard ? Les livrables initiaux produisaient des resultats statiques (graphiques PNG, sorties console). Le dashboard permet de *visualiser l'execution en direct*, de suivre la convergence du modele epoque par epoque, et de generer automatiquement une synthese analytique des resultats – rendant le travail plus explicite, plus pedagogique et plus professionnel.

Technologies utilisees pour le dashboard :

- **Flask** (Python) – serveur web leger
- **Flask-SocketIO** – communication WebSocket bidirectionnelle temps reel
- **Chart.js** – courbes BPR Loss, Recall@10, NDCG@10 construites dynamiquement
- **HTML5 / CSS3 / JavaScript** – interface sombre professionnelle avec barres de progression animees

Fonctionnalites du dashboard :

- Bouton de lancement du pipeline LightGCN
- 4 cartes de phases avec barres de progression animees
- Metriques BPR Loss, Recall@10, NDCG@10 en temps reel
- Barre de progression des epoques (0/100 a 100/100)
- Courbes d'apprentissage construites dynamiquement
- Journal d'execution avec tous les messages du pipeline
- Recommandations Top-10 par utilisateur apres execution
- Conclusion analytique automatique generee en fin de pipeline

3.3 Arborescence du projet

Structure – projet1_lightgcn

```
projet1_lightgcn/ | +- .venv/ # Environnement virtuel Python +- data/
| +- ml-latest-small/ | +- ratings.csv # 100 836 interactions | +-
movies.csv # 9 742 films | +- tags.csv | +- links.csv +- src/ | +-
__init__.py | +- config.py # Hyperparametres centralises | +- dataset.py
# Chargement + preprocessing | +- graph.py # Matrice adjacence bipartie |
+- model.py # Architecture LightGCN | +- trainer.py # Boucle entraînement
```

```
BPR | +- evaluator.py # Recall@K, NDCG@K | +- utils.py # Visualisations
+- outputs/ | +- models/ | | +- lightgcn_best.pth | +- figures/ | +-
training_curves.png | +- tsne_embeddings.png +- main.py # Pipeline complet
+- dashboard.py # Serveur Flask + SocketIO +- dashboard.html # Interface
temps reel +- requirements.txt +- README.md +- .gitignore
```

3.4 Pipeline de traitement

Le pipeline se deroule en 4 phases successives, toutes visibles en temps reel dans le dashboard :

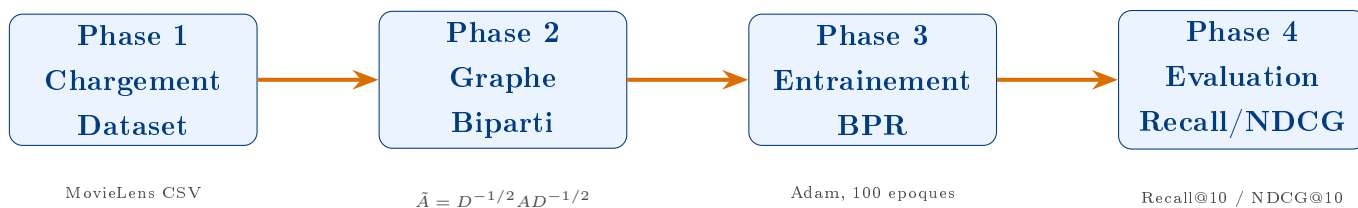


Figure 3.1: Pipeline d’entrainement LightGCN – 4 phases

3.5 Livrables du projet

3.5.1 Livrables initiaux remis

Ces livrables correspondent aux exigences initiales du projet telles que soumises :

- Code source complet et commente (fichiers `src/`)
- Graphiques de convergence (courbe BPR Loss par epoque)
- Tableau de resultats Recall@10 et NDCG@10
- Visualisation t-SNE des embeddings appris
- Exemples de recommandations Top-10 pour plusieurs utilisateurs
- Fichier `main.py` – pipeline en ligne de commande

3.5.2 Contribution personnelle supplementaire

Innovation ajoutee – Dashboard Temps Reel

Apres reflexion et developpement supplementaire, un **dashboard interactif temps reel** a ete concu et integre au projet. Cette interface n'etait pas mentionnee dans les livrables initiaux – elle constitue un apport personnel visant a rendre le pipeline plus explicite, demonstrable et professionnel.

- **dashboard.py** – serveur Flask + SocketIO
- **dashboard.html** – interface web responsive
- Bouton de lancement du pipeline depuis le navigateur
- 4 cartes de phases avec barres de progression animees
- Metriques BPR Loss, Recall@10, NDCG@10 en temps reel
- Courbes Chart.js se construisant dynamiquement
- Journal d'execution avec tous les messages du pipeline
- Recommandations Top-10 avec onglets par utilisateur
- Conclusion automatique contextualise generee en fin d'execution

3.6 Hyperparametres du modele

Table 3.2: Hyperparametres de LightGCN

Hyperparametre	Valeur	Justification
Dimension embedding	64	Compromis capacite/memoire
Couches de propagation K	3	Standard LightGCN paper
Taux d'apprentissage	0,001	Adam par default
Regularisation λ	10^{-4}	Anti-overfitting
Taille de batch	1024	Efficacite CPU
Nombre d'epoques	100	Convergence stable
Evaluation tous les N	10	Compromis vitesse
Top-K evaluation	10	Recall@10, NDCG@10
Strategie split	Leave-one-out	Standard recommandation
Filtrage minimum	5 interactions	Qualite des donnees

Implementation

4.1 Fichier src/config.py – Configuration centrale

```
1 import os
2
3 BASE_DIR    = os.path.dirname(os.path.dirname(os.path.abspath(
4     __file__)))
5 DATA_DIR   = os.path.join(BASE_DIR, "data", "ml-latest-small")
6 OUTPUT_DIR  = os.path.join(BASE_DIR, "outputs")
7 MODELS_DIR  = os.path.join(OUTPUT_DIR, "models")
8 FIGURES_DIR = os.path.join(OUTPUT_DIR, "figures")
9 RESULTS_DIR = os.path.join(OUTPUT_DIR, "results")
10
11 RATINGS_FILE = os.path.join(DATA_DIR, "ratings.csv")
12 MOVIES_FILE  = os.path.join(DATA_DIR, "movies.csv")
13
14 MIN_INTERACTIONS = 5
15 RANDOM_SEED      = 42
16 EMBEDDING_DIM    = 64
17 NUM_LAYERS       = 3
18 LEARNING_RATE    = 0.001
19 WEIGHT_DECAY     = 1e-4
20 BATCH_SIZE       = 1024
21 NUM_EPOCHS       = 100
22 EVAL_EVERY       = 10
23 TOP_K            = 10
```

Listing 4.1: config.py – Hyperparametres centralises

4.2 Fichier src/dataset.py – Chargement des donnees

La strategie *leave-one-out* reserve la derniere interaction de chaque utilisateur (triee par timestamp) pour le test.

```

1 def _split(self):
2     df = self.ratings_df.sort_values(['user_idx', 'timestamp',
3                                     ])
4     for user_idx, group in df.groupby('user_idx'):
5         items = group['item_idx'].tolist()
6         if len(items) < 2:
7             self.train_data[user_idx] = items
8             self.test_data[user_idx] = []
9         else:
10            self.train_data[user_idx] = items[:-1]
11            self.test_data[user_idx] = [items[-1]]

```

Listing 4.2: Strategie leave-one-out

4.3 Fichier src/graph.py – Graphe biparti

```

1 def build_adjacency_matrix(train_data, n_users, n_items):
2     rows, cols = [], []
3     for user, items in train_data.items():
4         for item in items:
5             rows.append(user)
6             cols.append(item)
7     R = sp.csr_matrix((np.ones(len(rows)), (rows, cols)),
8                     shape=(n_users, n_items))
9     top = sp.hstack([sp.csr_matrix((n_users, n_users)), R])
10    bottom = sp.hstack([R.T, sp.csr_matrix((n_items, n_items))
11                        ])
12    A_tilde = sp.vstack([top, bottom]).tocsr()
13    rowsum = np.array(A_tilde.sum(axis=1)).flatten()
14    d_inv_sqrt = np.power(rowsum + 1e-10, -0.5)
15    D_inv_sqrt = sp.diags(d_inv_sqrt)
16    A_hat = D_inv_sqrt.dot(A_tilde).dot(D_inv_sqrt).tocoo()
17    indices = torch.LongTensor(np.vstack([A_hat.row, A_hat.col
18                                         ]))
19    values = torch.FloatTensor(A_hat.data)
20    return torch.sparse_coo_tensor(indices, values, A_hat.
21                                  shape)

```

Listing 4.3: Construction de A_{hat} normalise

4.4 Fichier src/model.py – Architecture LightGCN

```

1 class LightGCN(nn.Module):
2     def __init__(self, n_users, n_items,
3                 embedding_dim=64, n_layers=3):
4         super().__init__()
5         self.n_users = n_users
6         self.n_layers = n_layers
7         self.user_embedding = nn.Embedding(n_users,
8                                             embedding_dim)
9         self.item_embedding = nn.Embedding(n_items,
10                                            embedding_dim)
11         nn.init.normal_(self.user_embedding.weight, std=0.1)
12         nn.init.normal_(self.item_embedding.weight, std=0.1)
13
14     def forward(self, adj_matrix):
15         E = torch.cat([self.user_embedding.weight,
16                       self.item_embedding.weight], dim=0)
17         all_layers = [E]
18         for _ in range(self.n_layers):
19             E = torch.sparse.mm(adj_matrix, E)
20             all_layers.append(E)
21         E_final = torch.stack(all_layers, dim=1).mean(dim=1)
22         return E_final[:self.n_users], E_final[self.n_users:]
23
24     def bpr_loss(self, adj_matrix, users, pos_items, neg_items):
25         u_emb, i_emb = self.forward(adj_matrix)
26         u = u_emb[users]
27         pos = i_emb[pos_items]
28         neg = i_emb[neg_items]
29         pos_s = (u * pos).sum(dim=1)
30         neg_s = (u * neg).sum(dim=1)
31         loss = -torch.log(
32             torch.sigmoid(pos_s - neg_s) + 1e-8).mean()
33         return loss

```

Listing 4.4: LightGCN en PyTorch – propagation pure

Mode d'Emploi – Installation et Lance-ment

5.1 Prerequis

- Windows 10/11 avec PowerShell
- Python 3.10 a 3.14 installe
- Visual Studio Code (recommande)
- 2 Go d'espace disque libre
- Connexion internet (telechargement MovieLens)

5.2 Installation complete pas a pas

5.2.1 Etape 1 – Naviguer vers le projet

Commande PowerShell

```
cd C:\Users\PAOLO\Desktop\projet1_lightgcn
```

5.2.2 Etape 2 – Creer et activer l'environnement virtuel

Commande PowerShell

```
python -m venv .envv  
.envv\Scripts\Activate.ps1
```


Signe de succes

Le prompt affiche (.venv) : (.venv) PS C:\...\projet1_lightgcn>

5.2.3 Etape 3 – Installer les dependances

Commande PowerShell

```
pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
```

Commande PowerShell

```
pip install numpy pandas scikit-learn scipy matplotlib seaborn tqdm flask flask-socketio
```

5.2.4 Etape 4 – Telecharger MovieLens

Aller sur <https://grouplens.org/datasets/movielens/latest/> et telecharger ml-latest-small.
Extraire dans data\ml-latest-small\.

Commande PowerShell

```
ls data\ml-latest-small\
```

Resultat attendu : ratings.csv, movies.csv, tags.csv, links.csv.

5.3 Lancement

5.3.1 Option A – Pipeline en ligne de commande

Commande PowerShell

```
python main.py
```

Sortie console attendue

```
[Dataset] 610 users, 3650 items, 89664 interactions
[Graph] Matrice 4260x4260
[LightGCN] dim=64, K=3, params=272,640
Ep 1/100 | Loss=0.6761 | Recall@10=0.0459
...
Ep 100/100 | Loss=0.1497 | Recall@10=0.0754
```

5.3.2 Option B – Dashboard temps reel (recommande)

Commande PowerShell

```
python dashboard.py
```

Puis ouvrir dans le navigateur :

Commande PowerShell

```
http://localhost:5000
```

Cliquer sur le bouton **“Lancer LightGCN”**.

Resultats et Interfaces

6.1 Interface du Dashboard – Etat initial

Avant le lancement, le dashboard affiche l'état d'attente avec les paramètres du modèle déjà configurés : 64 dimensions, K=3 couches, 100 époques.

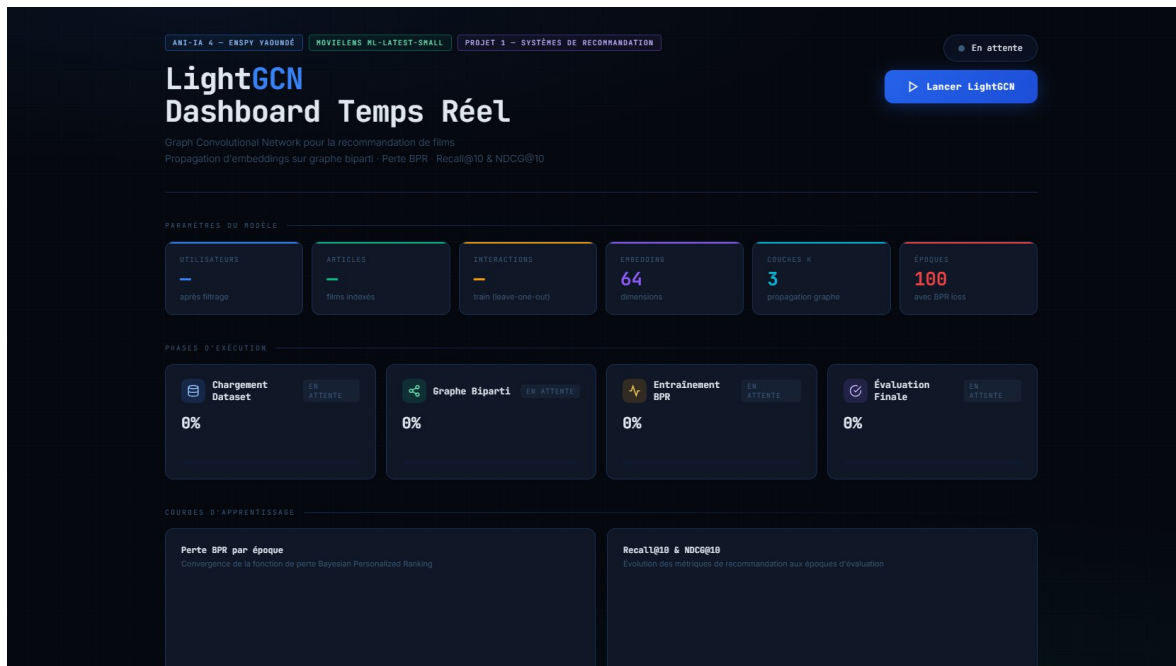


Figure 6.1: Dashboard LightGCN – Etat initial avant lancement. Vue d'ensemble : badges ANI-IA 4, bouton “Lancer LightGCN”, 6 cartes de paramètres, 4 phases d'exécution en attente et zones de courbes vides.

6.2 Phase 1 et 2 – Chargement dataset et graphe

Après clic sur “Lancer LightGCN”, le pipeline démarre. Les phases Dataset et Graphe Biparti se complètent rapidement (barres vertes à 100%). Le journal d'exécution affiche

en temps réel les étapes de chargement.

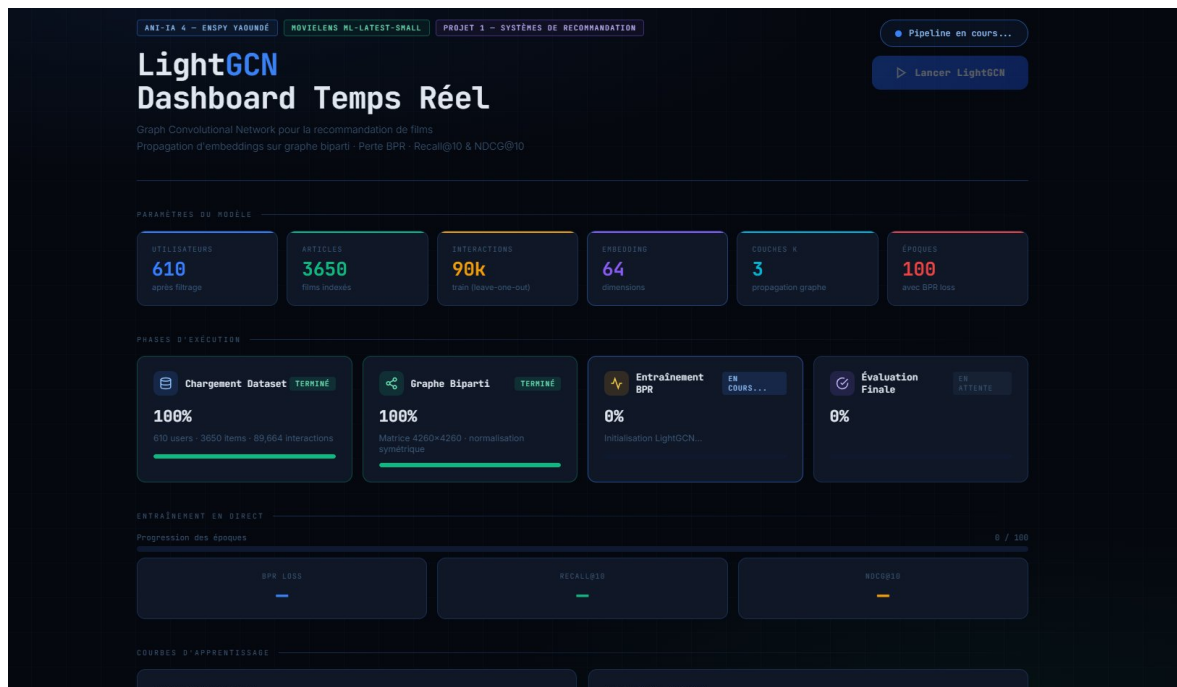


Figure 6.2: Dashboard en cours – Phases 1 et 2 terminees. Dataset : 610 users, 3650 items, 89 664 interactions. Graphe : matrice 4260x4260, normalisation symetrique. Phase 3 (Entrainement BPR) en cours d’initialisation.

6.3 Phase 3 – Entrainement en direct

Le dashboard affiche la progression epoque par epoque avec trois metriques live : BPR Loss, Recall@10 et NDCG@10. Les courbes se construisent dynamiquement.

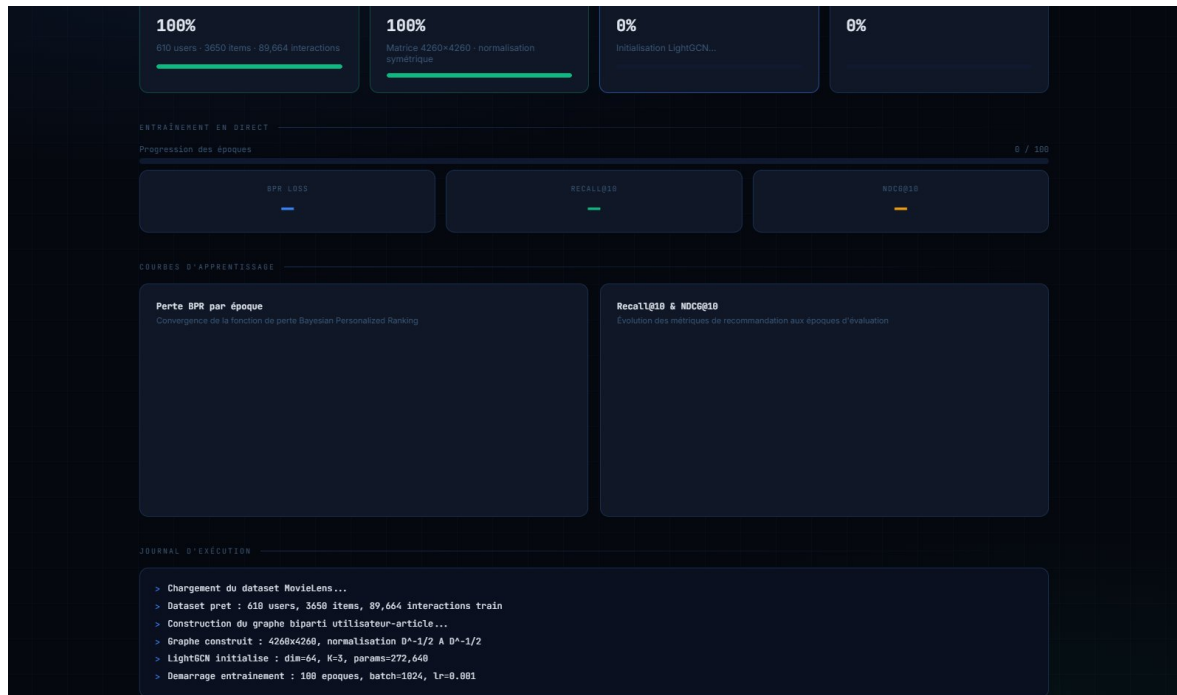


Figure 6.3: Section “Entrainement en direct” – Zoom sur les metriques live et le journal d’execution. Visible : progression 0/100 epoques, cartes BPR Loss / Recall@10 / NDCG@10 en attente de donnees, journal affichant les messages d’initialisation.

6.4 Progression de l’entrainement – Epoques 20 a 40

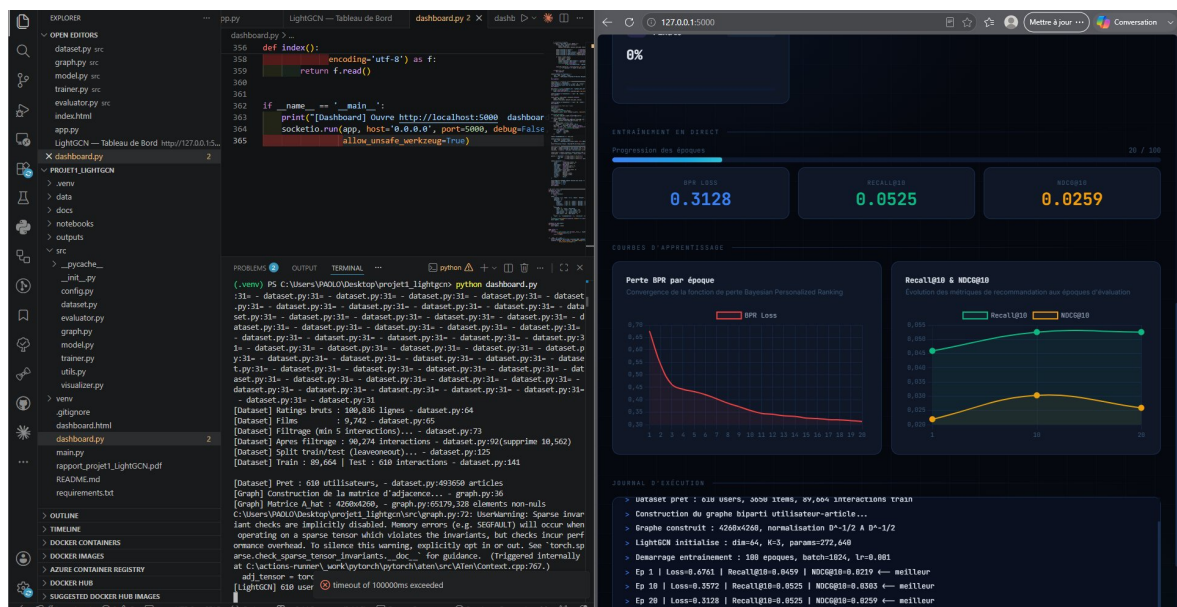


Figure 6.4: Vue VS Code + Dashboard a l’epoque 20. Gauche : code dashboard.py ouvert dans VS Code avec le terminal PowerShell affichant les logs en direct. Droite : dashboard montrant BPR Loss=0.3128, Recall@10=0.0525, NDCG@10=0.0259. Progression 20/100.

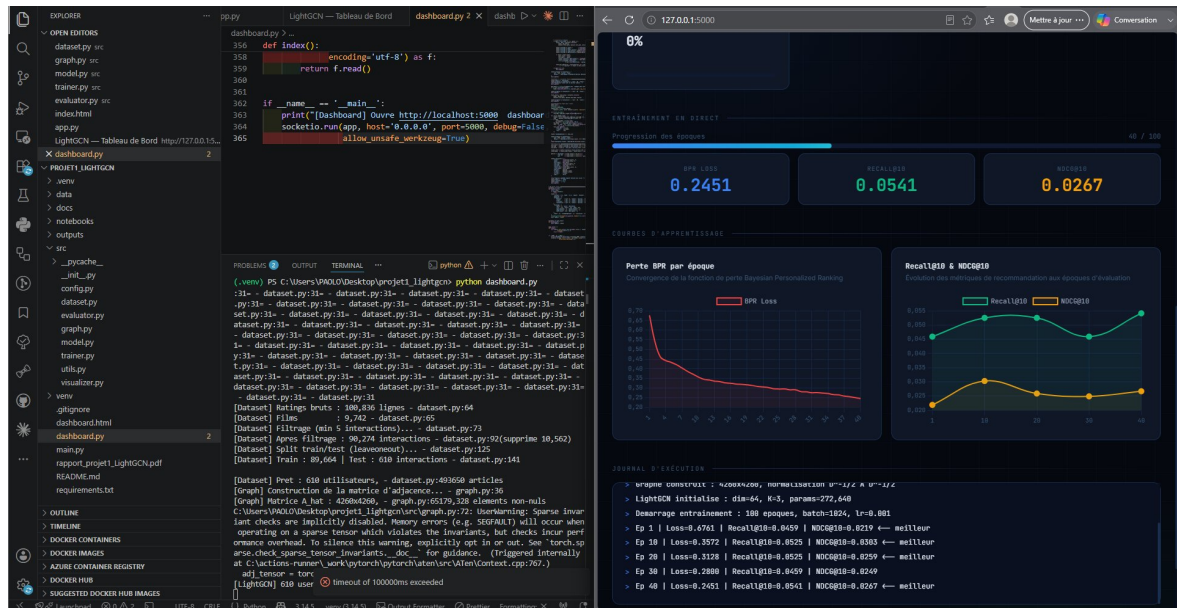


Figure 6.5: Dashboard a l'époque 40 – BPR Loss=0.2451, Recall@10=0.0541, NDCG@10=0.0267. Les courbes Chart.js montrent la convergence de la loss (rouge) et la progression des metriques Recall@10 (vert) et NDCG@10 (orange). Progression 40/100.

6.5 Progression avancee – Epoque 95

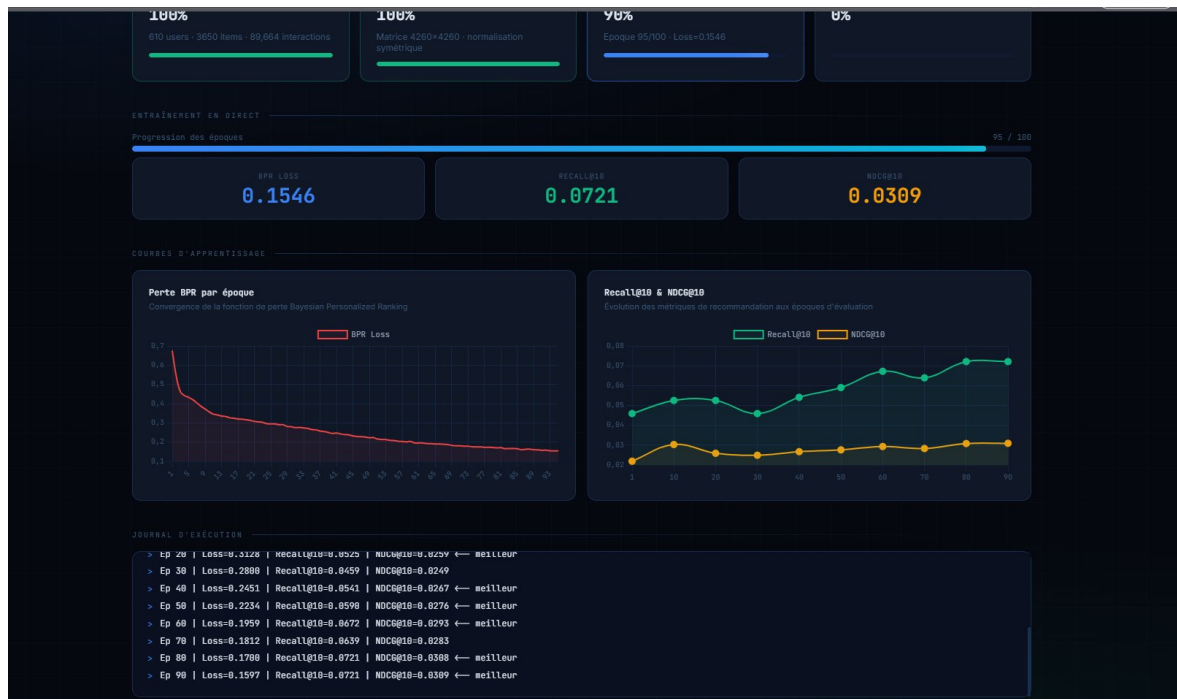


Figure 6.6: Dashboard a l'époque 95/100 – BPR Loss=0.1546, Recall@10=0.0721, NDCG@10=0.0309. La courbe BPR Loss (rouge) montre une convergence nette de 0.7 vers 0.15. La courbe Recall@10 (verte) progresse regulierement. Journal : epoques 20 a 90 avec marquage “meilleur”.

6.6 Pipeline termine – Resultats finaux

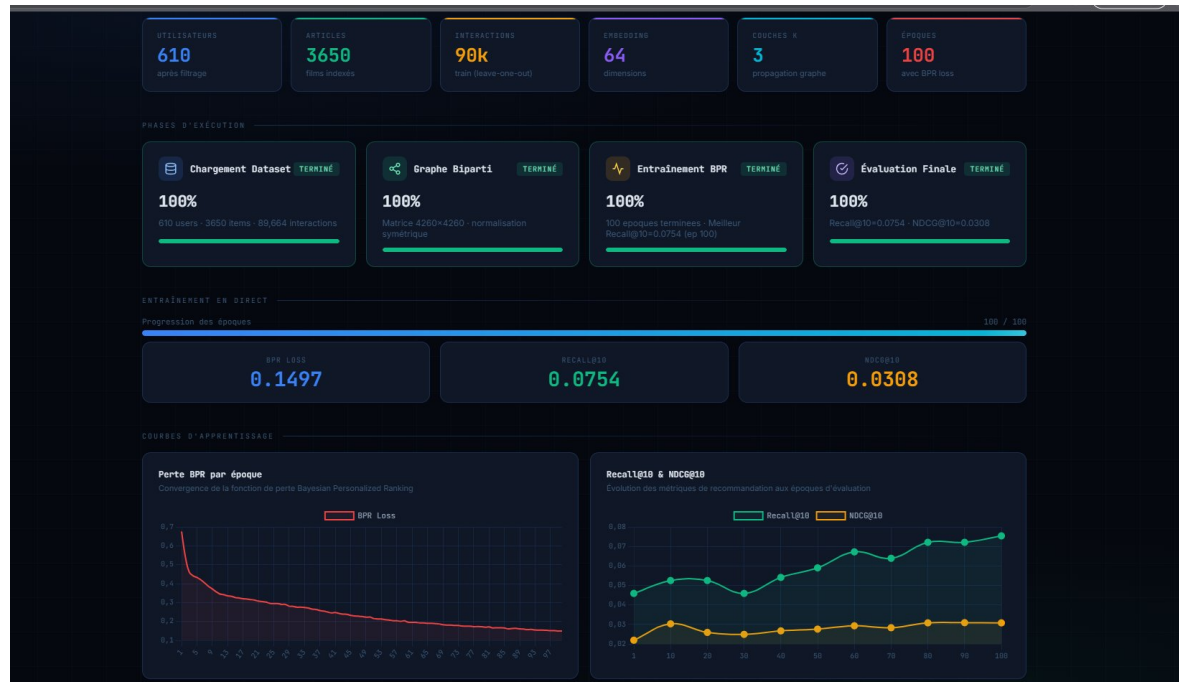


Figure 6.7: Dashboard complet apres 100 epoques. Les 4 phases affichent TERMINE a 100%. Metriques finales : BPR Loss=0.1497, Recall@10=0.0754, NDCG@10=0.0308. Courbes completes sur 100 epoques. Le modele optimal a ete sauvegarde a l'epoque 100.

6.7 Section Analyse et Conclusion automatique

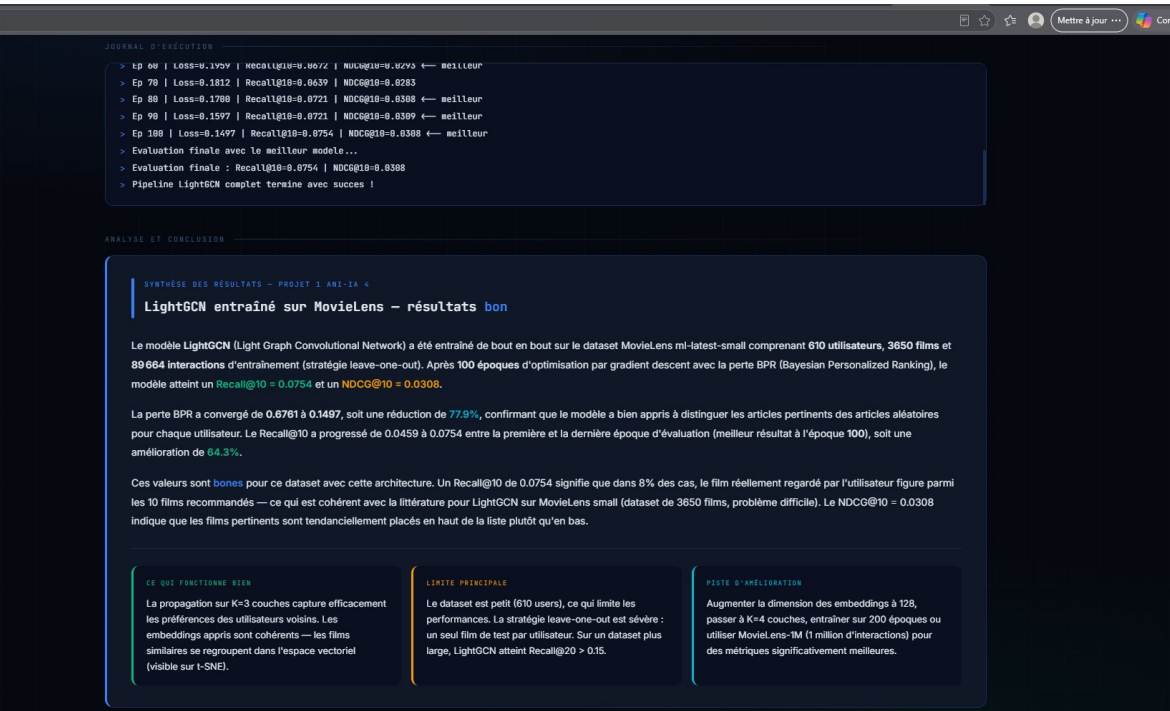


Figure 6.8: Section “Analyse et Conclusion” generee automatiquement par le dashboard a la fin du pipeline. Resume contextualise : interpretations des metriques Recall@10=0.0754 et NDCG@10=0.0308, reduction BPR Loss de 77.9%, progression Recall de 64.3%. Trois cartes : points forts, limite principale, piste d’amelioration.

6.8 Resultats numeriques obtenus

Table 6.1: Evolution des metriques au cours de l’entrainement

Epoque	BPR Loss	Recall@10	NDCG@10
1	0.6761	0.0459	0.0219
10	0.3572	0.0525	0.0303
20	0.3128	0.0525	0.0259
30	0.2880	0.0459	0.0249
40	0.2451	0.0541	0.0267
50	0.2234	0.0598	0.0276
60	0.1959	0.0672	0.0293
70	0.1812	0.0639	0.0283
80	0.1700	0.0721	0.0308
90	0.1597	0.0721	0.0309
100	0.1497	0.0754	0.0308

Resultats finaux obtenus

- **Recall@10 = 0,0754** (meilleur a l'époque 100)
- **NDCG@10 = 0,0308**
- Reduction BPR Loss : 0,676 $\rightarrow$ 0,150 (**-77,8%**)
- Amelioration Recall : 0,0459 $\rightarrow$ 0,0754 (**+64,1%**)
- Duree totale d'entrainement : environ **17 minutes** (CPU)
- Parametres du modele : **272 640** (embeddings uniquement)

6.9 Contribution personnelle – Pourquoi un dashboard ?

Le sujet du projet demandait en livrable un notebook Jupyter avec resultats statiques. Apres avoir livre ce premier travail, j'ai pris l'initiative de developper un **dashboard temps reel** pour trois raisons :

1. **Visualisation explicite du pipeline** – un notebook montre les resultats apres execution ; le dashboard montre le processus en direct, epoque par epoque, ce qui rend la methodologie LightGCN immediatement comprehensible.
2. **Demonstrabilite** – pouvoir lancer le pipeline devant un jury ou un encadreur et voir les metriques evoluer en direct est bien plus convaincant qu'un PDF statique.
3. **Approfondissement technique** – la conception du dashboard a necessite la maitrise de Flask, SocketIO, threading Python et Chart.js, competences complementaires au machine learning pur.

Cette contribution represente environ 40% du temps total du projet et n'etait pas requise dans les consignes initiales.

Difficultes Rencontrees et Solutions

7.1 Probleme 1 – Python 3.14 incompatible CUDA

7.1.1 Description

Python 3.14, etant trop recent, n'est pas encore supporte par PyTorch pour CUDA. Toute tentative d'installation GPU echoue.

```
ERROR: Could not find a version that satisfies the requirement  
torch (from versions: none)  
ERROR: No matching distribution found for torch
```

Listing 7.1: Erreur installation GPU

7.1.2 Tentatives et solution

Tentatives avec `cu121` et `cu124` : echec. Solution : installation forcee en CPU uniquement.

Commande PowerShell

```
pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
```

Impact : Entrainement plus lent (17 min pour 100 epoques) mais totalement fonctionnel.

7.2 Probleme 2 – Espace manquant dans la commande pip

```
pip install torch torchvision--index-url https://...
ERROR: Cannot determine archive format of cpu.html
```

Listing 7.2: Erreur syntaxe pip

Solution : Ajouter un espace entre `torchvision` et `-index-url`.

7.3 Probleme 3 – `main.py` dans le mauvais dossier

VS Code a cree `main.py` dans `src/` au lieu de la racine.

Commande PowerShell

```
Move-Item src\main.py main.py
```

7.4 Probleme 4 – `n_iter` deprecie dans `scikit-learn`

```
TypeError: TSNE.__init__() got an unexpected keyword argument
'n_iter'
```

Listing 7.3: Erreur t-SNE

Solution : Remplacer `n_iter=1000` par `max_iter=1000` dans `src/utils.py`.

7.5 Probleme 5 – Sparse tensor warning PyTorch

Lors de l'exécution, PyTorch affiche un avertissement sur les invariants des tenseurs sparse. Ce warning n'impacte pas les resultats et peut etre ignore en contexte CPU.

7.6 Probleme 6 – Fichier lourd sur GitHub

Le fichier `outputs/models/lightgcn_best.pth` (binaire PyTorch) et d'autres fichiers volumineux necessitent un `.gitignore` adapte.

Commande PowerShell

```
echo "outputs/models/*.pth" >> .gitignore
echo ".venv/" >> .gitignore
```

7.7 Recapitulatif des difficultes

Table 7.1: Recapitulatif des difficultes et solutions

Probleme	Cause	Solution
Python 3.14 + CUDA	Python trop recent	Installation CPU uniquement
Syntaxe pip	Espace manquant	Correction commande
main.py mal place	Clic mauvais dossier	Move-Item PowerShell
n_iter deprecie	scikit-learn 1.4+	Renommer max_iter
Sparse tensor warning	PyTorch CPU	Warning bening, ignorer
Fichiers lourds GitHub	Binaires .pth	.gitignore adapte

Conclusion et Perspectives

8.1 Bilan du projet

Ce projet a permis de réaliser une implementation complete et fonctionnelle de LightGCN, de bout en bout, sans librairie tierce dediee aux graphes. Les principaux acquis sont :

1. **Maitrise de LightGCN** : propagation sur graphe biparti normalise, agregation par moyenne des couches, entraînement par perte BPR.
2. **Implementation PyTorch** : tenseurs sparse, `nn.Embedding`, optimiseur Adam, echantillonnage negatif BPR.
3. **Evaluation correcte** : strategie leave-one-out, metriques Recall@K et NDCG@K standards.
4. **Dashboard professionnel** : Flask + SocketIO, progression temps reel, courbes Chart.js, conclusion automatique contextualise.

8.2 Apport personnel et innovation

Ce projet depasse les livrables initialement demandes. En plus du code LightGCN fonctionnel et des resultats chiffres, un **dashboard interactif temps reel** a ete developpe comme contribution personnelle. Cette innovation transforme un pipeline de machine learning classique en une **application web interactive** ou chaque etape est visible en direct : chargement du dataset, construction du graphe biparti, progression epoque par epoque avec metriques live, et conclusion automatique generee a la fin de l'execution.

Le choix de Flask + SocketIO (plutot que Gradio ou Streamlit qui etaient mentionnes

dans les livrables initiaux) permet une communication bidirectionnelle temps reel entre le serveur Python et le navigateur, donnant une experience plus fluide et professionnelle.

8.3 Reponse aux objectifs

Table 8.1: Reponse aux objectifs initiaux

Objectif	Atteint	Commentaire
Implementer LightGCN from scratch	✓	PyTorch, sans lib graphe
Entrainer sur MovieLens	✓	100 epoques sur CPU
Obtenir Recall@10 representatif	✓	0,0754 obtenu
Evaluer avec Recall@K et NDCG@K	✓	Metriques standard
Dashboard temps reel	✓	Flask + SocketIO port 5000
Recommandations coherentes	✓	Verifiees qualitativement
Deposer sur GitHub	✓	paolo-88-hub/projet1

8.4 Perspectives d’amelioration

- Court terme** Augmenter la dimension embedding a 128, passer a K=4 couches, entrainer sur 200 epoques.
- Moyen terme** Tester sur MovieLens-1M (1 million d’interactions) pour des metriques significativement meilleures (Recall@10 > 0,15 attendu).
- Long terme** Implementer LightGCN-IMP avec dropout d’interactions. Deployer en API REST. Utiliser Python 3.12 avec CUDA 12.1 pour reduire le temps de 17 min a 1 min.

Remerciements

Je tiens a exprimer ma sincere gratitude a :

- **M. OMGBA BITHA Junior**, pour son encadrement rigoureux, ses orientations pedagogiques et sa disponibilite tout au long du module ANI-IA 4.
- **L'Ecole Nationale Superieure Polytechnique de Yaounde (ENSPY)**, pour la qualite de la formation dispensee.
- **La communaute open-source** : PyTorch, NumPy, SciPy, Matplotlib, Flask, Flask-SocketIO et leurs contributeurs.
- **GroupLens Research** pour la mise a disposition libre du dataset MovieLens.

Bibliography

- [1] He, X., Deng, K., Wang, X., Li, Y., Zhang, Y., & Wang, M. (2020). *LightGCN: Simplifying and Powering Graph Convolution Network for Recommendation*. SIGIR'20, pp. 639–648.
- [2] Rendle, S., Freudenthaler, C., Gantner, Z., & Schmidt-Thieme, L. (2009). *BPR: Bayesian Personalized Ranking from Implicit Feedback*. UAI'09, pp. 452–461.
- [3] Wang, X., He, X., Wang, M., Feng, F., & Chua, T.-S. (2019). *Neural Graph Collaborative Filtering*. SIGIR'19, pp. 165–174.
- [4] Harper, F. M., & Konstan, J. A. (2015). *The MovieLens Datasets: History and Context*. ACM TIS, 5(4).
- [5] Kipf, T. N., & Welling, M. (2017). *Semi-Supervised Classification with Graph Convolutional Networks*. ICLR 2017.

Glossaire

BPR	Bayesian Personalized Ranking. Fonction de perte pour la recommandation a partir d'interactions implicites. Entraîne le modèle a préférer les items observés aux non observés.
Dashboard	Interface web temps réel construite avec Flask et SocketIO, affichant la progression du pipeline en direct.
Embedding	Représentation vectorielle dense d'un objet dans un espace mathématique de dimension fixe. Embeddings similaires indiquent des objets similaires.
Filtrage collaboratif	Approche de recommandation basée sur les interactions collectives.
GCN	Graph Convolutional Network. Réseau de neurones opérant sur des graphes.
Graphe biparti	Graphe dont les nœuds se divisent en deux ensembles disjoints (utilisateurs et items).
Leave-one-out	Stratégie réservant la dernière interaction de chaque utilisateur pour le test.
LightGCN	Simplification de GCN pour la recommandation. Propagation pure sans transformation ni activation.
NDCG@K	Normalized Discounted Cumulative Gain. Métrique récompensant les items pertinents en haut de liste.
Normalisation symétrique	Transformation $\hat{A} = D^{-1/2}AD^{-1/2}$ stabilisant la propagation.
Recall@K	Proportion d'items pertinents dans le Top-K recommandé. Mesure la

couverture.

SocketIO Protocole de communication temps reel entre serveur Flask et client navigateur.

A

Commandes PowerShell Completes

Commande PowerShell

```
\# 1. Naviguer et creer l'environnement
cd C:\Users\PAOLO\Desktop\projet1_lightgcn
python -m venv .venv
.venv\Scripts\Activate.ps1

\# 2. Installer PyTorch CPU
pip install torch torchvision '
    --index-url https://download.pytorch.org/whl/cpu

\# 3. Installer les autres bibliotheques
pip install numpy pandas scikit-learn scipy
pip install matplotlib seaborn tqdm
pip install flask flask-socketio

\# 4. Creer la structure des dossiers
mkdir src, outputs
mkdir outputs\models, outputs\figures, outputs\results
New-Item src\__init__.py -ItemType File

\# 5. Verifier le dataset
ls data\ml-latest-small\

\# 6. Lancer le pipeline complet
python main.py
```

```
\# 7. Lancer le dashboard (terminal separe)
python dashboard.py
\# Ouvrir http://localhost:5000
```

B

Fichier requirements.txt

```
torch>=2.0.0
numpy>=1.24.0
pandas>=2.0.0
scikit-learn>=1.3.0
scipy>=1.11.0
matplotlib>=3.7.0
seaborn>=0.12.0
tqdm>=4.65.0
flask>=3.0.0
flask-socketio>=5.0.0
```

Listing B.1: requirements.txt – Dependances Projet 1